

Spring Security

Authentification, JWT, OAuth2 / OIDC, Keycloak

Formateur : Haithem Mihoubi

Module : 3 / 3

Durée : 5 jours (35 heures)

Niveau : Débutant → Avancé

Public : développeurs Java / Spring Boot

Pré-requis : Java 17+, bases de Spring Boot et des API REST

Sommaire

Avant de commencer	3
Jour 1 — Fondamentaux & architecture Spring Security 6.....	4
1.1 Authentification vs autorisation	4
1.2 Stateless vs stateful	4
1.3 Hachage des mots de passe	5
1.4 Attaques courantes & parades	6
1.5 Architecture Spring Security 6.....	6
1.6 Hands-On — Premier projet sécurisé	7
Quiz — Jour 1	8
Jour 2 — Authentification avancée & autorisation (RBAC)	9
2.1 Mécanismes d'authentification	9
2.2 UserDetailsService & PasswordEncoder	9
2.3 Rôles, autorités & permissions	10
2.4 Autorisation au niveau méthode.....	10
2.5 Atelier — Modèle Rôle/Permission & endpoints protégés.....	11
Quiz — Jour 2	12
Jour 3 — JWT & sécurité des API REST	13
3.1 Pourquoi JWT ? Structure	13
3.2 Access token vs refresh token	14
3.3 Implémentation JWT dans Spring Security 6.....	14
3.4 Gestion des erreurs	16
3.5 Atelier — API REST sécurisée par JWT.....	17
Quiz — Jour 3	17
Jour 4 — OAuth2, OpenID Connect & Keycloak.....	18
4.1 Concepts OAuth2 / OIDC.....	18
4.2 Spring Security + OAuth2.....	19
4.3 Keycloak	20
4.4 Atelier — Login OAuth2 + front protégé	21
Quiz — Jour 4	21
Jour 5 — Sécurité avancée, bonnes pratiques & TP final	22
5.1 Sécurisation réelle d'une application.....	22
5.2 Stratégies avancées	23
5.3 Grand TP final	24
Évaluation finale du Module 3	25

Avant de commencer

Ce module est un **mini-projet évolutif sur 5 jours** : on construit une API sécurisée qui grossit chaque jour (auth basique → RBAC → JWT → Keycloak → durcissement). Le fil rouge est la **QuickBite API** (gestion d'utilisateurs, commandes, console admin) — chaque notion est appliquée immédiatement sur ce projet.

Préparer votre poste

JDK 17+ (idéalement 21), Maven ou Gradle, un IDE (IntelliJ/VS Code), **Postman** (ou `curl` / `httpie`), **Docker** (pour Keycloak et PostgreSQL au Jour 4). Générez le squelette sur start.spring.io (dépendances : Web, Security, JPA, PostgreSQL, Validation).

Versions de référence

Spring Boot 3.x, Spring Security 6.x (Java 17+). Attention : la configuration a beaucoup changé par rapport à Spring Security 5 — voir 1.5.

Jour 1 — Fondamentaux & architecture Spring Security 6

🎯 Objectifs du Jour 1

- Distinguer authentification et autorisation.
- Choisir entre une sécurité stateful et stateless.
- Hacher correctement des mots de passe (BCrypt/Argon2).
- Reconnaître les attaques courantes et leurs parades.
- Comprendre l'architecture de Spring Security 6 et créer un premier projet sécurisé.

1.1 Authentification vs autorisation

	Authentification (AuthN)	Autorisation (AuthZ)
Question	« Qui es-tu ? »	« As-tu le droit de faire ceci ? »
Vérifie	Identité (login/mot de passe, token, certificat)	Permissions (rôles, scopes)
Vient	En premier	Après l'authentification
Exemple	Se connecter à QuickBite	Accéder à la console admin

! Ne jamais confondre les deux

Un utilisateur **authentifié** n'est pas pour autant **autorisé**. Les deux contrôles sont distincts et tous deux obligatoires.

1.2 Stateless vs stateful

- **Stateful (session)** : le serveur garde une **session** en mémoire ; le client renvoie un cookie `JSESSIONID`. Simple, mais ne *scale* pas bien (sessions à répliquer) et sensible au CSRF.
- **Stateless (token)** : le serveur ne stocke rien ; le client porte un **token** (JWT) à chaque requête. Idéal pour API REST et microservices, friendly au scaling.

Critère	Stateful	Stateless (JWT)
Stockage serveur	Session	Aucun
Scalabilité horizontale	Difficile	Facile
Révocation immédiate	Facile (invalider la session)	Difficile (token valide jusqu'à expiration)
Cas d'usage	App web classique	API REST, mobile, microservices

1.3 Hachage des mots de passe

On ne **stocke jamais** un mot de passe en clair, ni chiffré de façon réversible : on stocke un **hash** lent et salé.

```
@Bean
public PasswordEncoder passwordEncoder() {
    // BCrypt avec un coût (work factor) de 12
    return new BCryptPasswordEncoder(12);
}
```

Algorithme	Caractéristique
BCrypt	Standard éprouvé, sel intégré, coût ajustable
Argon2	Lauréat du Password Hashing Competition, résistant GPU/mémoire (recommandé pour le neuf)
PBKDF2	Largement supporté, conforme FIPS
~~MD5 / SHA-1~~	Interdits : trop rapides, cassables

```
// Choisir/migrer les encoders dynamiquement
@Bean
PasswordEncoder passwordEncoder() {
    return PasswordEncoderFactories.createDelegatingPasswordEncoder();
    // stocke un préfixe {bcrypt}, {argon2}... -> migration transparente
}
```

⚠ Le « sel » et le coût

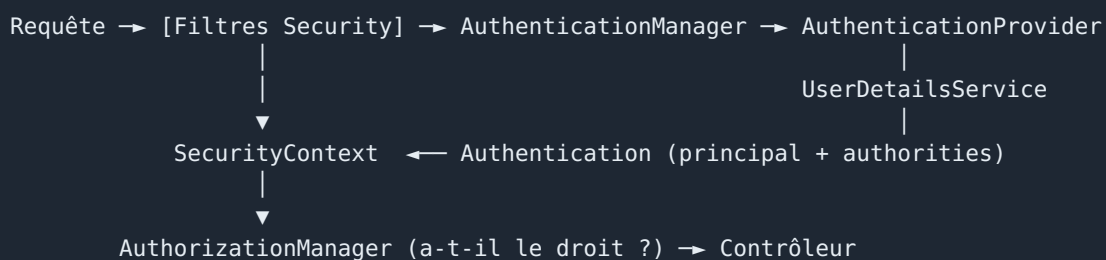
Le **sel** (aléatoire par mot de passe) empêche les rainbow tables ; BCrypt l'intègre automatiquement. Le **coût** rend le hachage volontairement lent pour freiner le brute force. Augmentez-le avec le matériel (12+ aujourd'hui).

1.4 Attaques courantes & parades

Attaque	Principe	Parade
Brute force	Essayer des milliers de mots de passe	Hash lent, rate limiting, logout, MFA
MITM (Man in the Middle)	Intercepter le trafic	HTTPS/TLS partout , HSTS
XSS	Injecter du JS dans une page	Échapper les sorties, CSP, cookies <code>HttpOnly</code>
CSRF	Forcer une requête authentifiée à l'insu de l'utilisateur	Token anti-CSRF, <code>SameSite</code> , API stateless
Injection SQL	Injecter du SQL via une entrée	Requêtes paramétrées / ORM
Credential stuffing	Réutiliser des fuites de mots de passe	MFA, détection d'anomalies

1.5 Architecture Spring Security 6

Spring Security s'insère comme une **chaîne de filtres** (`SecurityFilterChain`) devant votre application. Chaque requête traverse ces filtres avant d'atteindre votre contrôleur.



Composant	Rôle
<code>SecurityFilterChain</code>	Déclare les règles (URLs publiques/protégées, login, CSRF...)
<code>AuthenticationManager</code>	Orchestre l'authentification
<code>AuthenticationProvider</code>	Vérifie réellement les identifiants
<code>UserDetailsService</code>	Charge l'utilisateur (depuis la base)
<code>UserDetails</code> / <code>GrantedAuthority</code>	L'utilisateur et ses rôles/autorités
<code>SecurityContext</code>	Stocke l'utilisateur authentifié pour la requête

Le changement majeur vs Spring Security 5

! **WebSecurityConfigurerAdapter est supprimé**

L'ancien modèle (étendre `WebSecurityConfigurerAdapter`, override `configure(HttpSecurity)`) **n'existe plus** en Spring Security 6. On déclare désormais un **bean** `SecurityFilterChain` et on utilise la **configuration par lambda**.

```
// Spring Security 6 – configuration moderne
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/", "/public/**").permitAll()
                .requestMatchers("/admin/**").hasRole("ADMIN")
                .anyRequest().authenticated()
            )
            .httpBasic(Customizer.withDefaults())
            .formLogin(Customizer.withDefaults());
        return http.build();
    }

    @Bean
    PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }
}
```

1.6 Hands-On — Premier projet sécurisé

Atelier 1 — Durée 1 h 30

Mission : créer un projet Spring Boot 3, ajouter Spring Security, et observer le comportement par défaut, puis définir des règles d'accès.

Étapes : 1. (15 min) Générer le projet (Web + Security) ; lancer et constater la page de login par défaut + le mot de passe généré en console. 2. (20 min) Ajouter un contrôleur `/` (public) et `/private` (protégé) ; configurer la `SecurityFilterChain`. 3. (25 min) Définir **deux utilisateurs en mémoire** (`user`, `admin`) avec rôles ; tester l'accès à `/admin`. 4. (20 min, optionnel) Activer **HTTPS** avec un certificat auto-signé (`keytool`). 5. (10 min) Restitution.

Utilisateurs en mémoire (pour démarrer vite) :

```
@Bean
UserDetailsService users(PasswordEncoder encoder) {
    UserDetails user = User.withUsername("user")
        .password(encoder.encode("user123")).roles("USER").build();
    UserDetails admin = User.withUsername("admin")
        .password(encoder.encode("admin123")).roles("ADMIN").build();
    return new InMemoryUserDetailsManager(user, admin);
}
```

⚠ Le piège du préfixe `ROLE_`

Observez le mot de passe généré par défaut affiché dans la console au démarrage. Et retenez bien la différence, source d'erreurs n°1 : `hasRole("ADMIN")` ajoute **implicitement** le préfixe et attend donc une autorité `ROLE_ADMIN`, alors que `hasAuthority("ROLE_ADMIN")` exige la chaîne exacte. Mélanger les deux est la cause classique des `403` inexplicables.

Quiz — Jour 1

1. AuthN vs AuthZ ?
2. Pourquoi du stateless pour une API REST ?
3. Pourquoi BCrypt plutôt que SHA-256 pour un mot de passe ?
4. Citez deux parades au CSRF.
5. Quel composant remplace `WebSecurityConfigurerAdapter` ?

Jour 2 — Authentification avancée & autorisation (RBAC)

🎯 Objectifs du Jour 2

- Mettre en place une authentification basée sur une base de données.
- Implémenter un `UserDetailsService` personnalisé.
- Modéliser rôles, autorités et permissions.
- Protéger des méthodes avec `@PreAuthorize`, `@Secured` et des expressions.

2.1 Mécanismes d'authentification

- **Basic Auth** : identifiants en en-tête `Authorization: Basic ...` (base64). Simple, à n'utiliser qu'en HTTPS.
- **Form login** : formulaire HTML + session. Pour applications web classiques.
- **Token (Bearer/JWT)** : `Authorization: Bearer <token>` (vu au Jour 3).

2.2 UserDetailsService & PasswordEncoder

On remplace les utilisateurs en mémoire par des utilisateurs **persistés** en base.

Entité JPA :

```
@Entity
@Table(name = "users")
public class UserEntity {
    @Id @GeneratedValue
    private Long id;
    @Column(unique = true, nullable = false)
    private String username;
    @Column(nullable = false)
    private String password;           // hash BCrypt
    @ManyToMany(fetch = FetchType.EAGER)
    private Set<RoleEntity> roles = new HashSet<>();
    // getters/setters
}
```

`UserDetailsService` personnalisé :

```

@Service
public class JpaUserDetailsService implements UserDetailsService {

    private final UserRepository repo;
    public JpaUserDetailsService(UserRepository repo) { this.repo = repo; }

    @Override
    public UserDetails loadUserByUsername(String username) {
        UserEntity u = repo.findByUsername(username)
            .orElseThrow(() -> new UsernameNotFoundException(username));
        var authorities = u.getRoles().stream()
            .map(r -> new SimpleGrantedAuthority("ROLE_" + r.getName()))
            .collect(Collectors.toList());
        return new org.springframework.security.core.userdetails.User(
            u.getUsername(), u.getPassword(), authorities);
    }
}

```



Message d'erreur volontairement flou

À l'échec d'authentification, renvoyez un message générique (« identifiants invalides ») — sans préciser si c'est le login ou le mot de passe qui est faux — pour ne pas aider l'attaquant à énumérer les comptes.

2.3 Rôles, autorités & permissions

- **Rôle** : un regroupement grossier (`ROLE_ADMIN` , `ROLE_USER`).
- **Autorité / Permission** : un droit fin (`order:read` , `order:cancel`).

Le modèle **RBAC** mature associe : *Utilisateur* → *Rôles* → *Permissions*. On contrôle alors par **permission** (fin) plutôt que par rôle (grossier), ce qui évite de réécrire le code quand les rôles changent.

Utilisateur	—*..*— Rôle	—*..*— Permission
alice	ADMIN	order:read, order:delete, user:manage
bob	CLIENT	order:read, order:create

2.4 Autorisation au niveau méthode

Activer la sécurité de méthode :

```

@Configuration
@EnableMethodSecurity // active @PreAuthorize/@PostAuthorize/@Secured
public class MethodSecurityConfig { }

```

```

@RestController
@RequestMapping("/orders")
public class OrderController {

    @GetMapping
    @PreAuthorize("hasAuthority('order:read')")
    public List<OrderDto> all() { ... }

    @DeleteMapping("/{id}")
    @PreAuthorize("hasRole('ADMIN')")
    public void delete(@PathVariable Long id) { ... }

    // Expression : l'utilisateur ne lit que SES commandes
    @GetMapping("/{id}")
    @PreAuthorize("hasRole('ADMIN') or #username == authentication.name")
    public OrderDto getOne(@PathVariable Long id,
                           @RequestParam String username) { ... }
}

```

Annotation	Quand l'évaluer	Remarque
<code>@PreAuthorize</code>	Avant la méthode	Le plus utilisé, expressions SpEL riches
<code>@PostAuthorize</code>	Après (sur le retour)	Filtrer selon l'objet renvoyé
<code>@Secured</code>	Avant	Plus ancien, rôles uniquement
<code>@RolesAllowed</code>	Avant	Standard Jakarta équivalent

2.5 Atelier — Modèle Rôle/Permission & endpoints protégés

Atelier 2 — Durée 2 h 30

Mission : persister les utilisateurs en base, brancher un `UserDetailsService` JPA, modéliser Rôles/Permissions, et protéger les endpoints REST selon les profils.

Étapes : 1. (40 min) Entités `UserEntity`, `RoleEntity` (+ permissions) et repositories ; jeu de données initial (admin + client). 2. (35 min) Implémenter `JpaUserDetailsService` ; configurer `AuthenticationProvider` (DAO + `PasswordEncoder`). 3. (35 min) Protéger : `/orders` (lecture pour CLIENT), `/admin/**` (ADMIN), suppression réservée ADMIN. 4. (30 min) Tester avec Postman : 200 / 401 / 403 selon les profils. 5. (20 min) Restitution + relecture des règles.

Critères de réussite :

- Un CLIENT lit ses commandes mais reçoit **403** sur `/admin`.
- Un mauvais mot de passe → **401**.
- La suppression d'une commande par un CLIENT → **403**.

⚠️ 401 ou 403 ? Ne les confondez pas

Distinction essentielle : **401 (non authentifié)** = « je ne sais pas qui tu es » (token absent ou invalide) ; **403 (authentifié mais non autorisé)** = « je sais qui tu es, mais tu n'as pas le droit ». Et rappel du piège du préfixe : `hasRole('ADMIN')` attend une autorité `ROLE_ADMIN` en base.

Quiz — Jour 2

1. Différence rôle vs permission ?
 2. Que charge `loadUserByUsername` ?
 3. `@PreAuthorize` vs `@PostAuthorize` ?
 4. Quel code HTTP pour « authentifié mais pas le droit » ?
 5. Pourquoi un message d'erreur d'auth générique ?
-

Jour 3 — JWT & sécurité des API REST

🎯 Objectifs du Jour 3

- Expliquer la structure d'un JWT et ses avantages/limites.
- Distinguer access token et refresh token.
- Implémenter l'émission et la validation de JWT dans Spring Security 6.
- Gérer proprement les erreurs d'authentification/autorisation.

3.1 Pourquoi JWT ? Structure

Un **JWT** (JSON Web Token) est un jeton **auto-porteur** : il contient les informations d'identité et est **signé**, donc vérifiable sans appeler la base à chaque requête. Idéal pour le **stateless**.

```
header.payload.signature      (3 parties encodées base64url, séparées par des points)
eyJhbGciOiJIUzI1NiJ9 . eyJzdWIiOiJhbGljZSIsInJvbGUiOiJBRElJTj9 . 3xT9...signature
```

Partie	Contenu	Exemple
Header	Algorithme, type	<code>{"alg": "HS256", "typ": "JWT"}</code>
Payload	Claims (sub, exp, rôles...)	<code>{"sub": "alice", "role": "ADMIN", "exp": ...}</code>
Signature	HMAC/RSA du header+payload	garantit l' intégrité

! Un JWT n'est PAS chiffré

Le payload est seulement **encodé** (base64), donc **lisible** par quiconque. N'y mettez jamais de données sensibles (mot de passe, secret). La signature garantit qu'il n'a pas été **modifié**, pas qu'il est secret.

JWT vs session

	Session (JSESSIONID)	JWT
État serveur	Oui	Non
Scalabilité	Moins bonne	Excellente
Révocation	Immédiate	Difficile (→ durée courte + blacklist)
Cross-domaine / mobile	Limité	Naturel

3.2 Access token vs refresh token

- **Access token** : courte durée (5–15 min), envoyé à chaque requête. S'il fuit, le risque est limité dans le temps.
- **Refresh token** : longue durée (jours/semaines), stocké de façon sécurisée, sert **uniquement** à obtenir un nouvel access token.

```
Login → access (15 min) + refresh (7 j)
      |
      |— requêtes API avec l'access token
      |
      |— access expiré → POST /auth/refresh (refresh token) → nouvel access token
```

⚠ Rotation et stockage des refresh tokens

Appliquez la **rotation** (un refresh utilisé est invalidé et remplacé) et stockez-les en base pour pouvoir les **révoquer**. Côté front, préférez un cookie `HttpOnly` + `SameSite` au `localStorage` (sensible au XSS).

3.3 Implémentation JWT dans Spring Security 6

💡 Deux approches

(A) Resource Server OAuth2 de Spring (recommandé, peu de code) avec `spring-boot-starter-oauth2-resource-server` et la validation de JWT intégrée. **(B) Filtre maison** (pédagogique) qui montre la mécanique. On présente les deux ; l'atelier suit l'approche maison pour comprendre, puis on montre l'approche standard.

Service d'émission de token (JWT)

```
@Service
public class JwtService {
    private final SecretKey key = Keys.hmacShaKeyFor(
        System.getenv("JWT_SECRET").getBytes(StandardCharsets.UTF_8));

    public String generateAccess(UserDetails user) {
        return Jwts.builder()
            .subject(user.getUsername())
            .claim("roles", user.getAuthorities().stream()
                .map(GrantedAuthority::getAuthority).toList())
            .issuedAt(new Date())
            .expiration(new Date(System.currentTimeMillis() + 900_000)) // 15 min
            .signWith(key)
            .compact();
    }

    public Jws<Claims> parse(String token) {
        return Jwts.parser().verifyWith(key).build().parseSignedClaims(token);
    }
}
```

Filtre d'authentification JWT (à chaque requête)

```
@Component
public class JwtAuthFilter extends OncePerRequestFilter {

    private final JwtService jwt;
    public JwtAuthFilter(JwtService jwt) { this.jwt = jwt; }

    @Override
    protected void doFilterInternal(HttpServletRequest req,
        HttpServletResponse res, FilterChain chain)
        throws ServletException, IOException {
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            try {
                Claims claims = jwt.parse(header.substring(7)).getPayload();
                var roles = ((List<?>) claims.get("roles")).stream()
                    .map(r -> new SimpleGrantedAuthority(r.toString())).toList();
                var auth = new UsernamePasswordAuthenticationToken(
                    claims.getSubject(), null, roles);
                SecurityContextHolder.getContext().setAuthentication(auth);
            } catch (JwtException e) {
                SecurityContextHolder.clearContext(); // token invalide -> non authentifié
            }
        }
        chain.doFilter(req, res);
    }
}
```

Configuration stateless

```
@Bean
SecurityFilterChain api(HttpSecurity http, JwtAuthFilter jwtFilter) throws Exception {
    http
        .csrf(csrf -> csrf.disable()) // API stateless -> CSRF non pertinent
        .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/auth/**").permitAll()
            .requestMatchers("/admin/**").hasRole("ADMIN")
            .anyRequest().authenticated())
        .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class);
    return http.build();
}
```

Approche standard (Resource Server) — pour comparaison

```
http.oauth2ResourceServer(oauth2 -> oauth2.jwt(Customizer.withDefaults()));
```

```
# application.yml - validation automatique via JWKS
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          jwk-set-uri: https://idp.exemple.com/.well-known/jwks.json
```

3.4 Gestion des erreurs

```
// 401 : non authentifié
@Component
public class JwtEntryPoint implements AuthenticationEntryPoint {
    public void commence(HttpServletRequest req, HttpServletResponse res,
        AuthenticationException ex) throws IOException {
        res.sendError(HttpServletResponse.SC_UNAUTHORIZED, "Authentification requise");
    }
}

// 403 : authentifié mais non autorisé
@Component
public class RestAccessDeniedHandler implements AccessDeniedHandler {
    public void handle(HttpServletRequest req, HttpServletResponse res,
        AccessDeniedException ex) throws IOException {
        res.sendError(HttpServletResponse.SC_FORBIDDEN, "Accès refusé");
    }
}
```

```
http.exceptionHandling(e -> e
    .authenticationEntryPoint(entryPoint)
    .accessDeniedHandler(accessDeniedHandler));
```


3.5 Atelier — API REST sécurisée par JWT

Atelier 3 — Durée 3 h

Mission : implémenter une authentification JWT complète sur la QuickBite API : login, refresh, endpoints protégés, gestion d'erreurs, tests Postman.

Endpoints à livrer :

```
POST /auth/login      -> { accessToken, refreshToken }
POST /auth/refresh    -> { accessToken }
GET  /users/me        -> profil (authentifié)
GET  /admin/users     -> liste (ADMIN uniquement)
```

Étapes : 1. (40 min) `JwtService` (génération/validation) + secret via variable d'environnement. 2. (40 min) `/auth/login` : authentifier puis émettre access + refresh. 3. (40 min) `JwtAuthFilter` + configuration stateless. 4. (30 min) `/auth/refresh` avec rotation ; stockage des refresh tokens en base. 5. (30 min) `EntryPoint` (401) + `AccessDeniedHandler` (403). 6. (20 min) Tests Postman : créer une collection + variables d'environnement (token).

Critères de réussite (DoD) :

- Login renvoie deux tokens ; `/users/me` fonctionne avec le Bearer.
- `/admin/users` → 403 pour un non-admin.
- Token expiré → 401 propre ; refresh renvoie un nouvel access token.

Les pièges classiques du JWT

Trois erreurs fréquentes : oublier `STATELESS` (Spring recrée alors des sessions à votre insu), placer le filtre JWT au mauvais endroit dans la chaîne, et utiliser un secret trop court pour HS256 (il faut ≥ 256 bits). Astuce de productivité : stockez le token dans une **variable Postman** via un script de test — vous gagnerez un temps énorme pour la suite des ateliers.

Quiz — Jour 3

1. Un JWT est-il chiffré ?
2. Access vs refresh token ?
3. Pourquoi `SessionCreationPolicy.STATELESS` ?
4. Que garantit la signature d'un JWT ?
5. Où placer le `JwtAuthFilter` dans la chaîne ?

Jour 4 — OAuth2, OpenID Connect & Keycloak

🎯 Objectifs du Jour 4

- Expliquer OAuth2, OIDC et les principaux flows (dont PKCE).
- Distinguer access token et ID token.
- Configurer Spring Security en client OAuth2 et en resource server.
- Installer et paramétrer Keycloak ; sécuriser une API et un front.

4.1 Concepts OAuth2 / OIDC

- **OAuth2** = protocole d'**autorisation déléguée** (« autoriser une app à accéder à des ressources en mon nom », sans lui donner mon mot de passe).
- **OpenID Connect (OIDC)** = couche d'**authentification** au-dessus d'OAuth2 (« qui est l'utilisateur ») — ajoute l'**ID token**.

Les acteurs

Rôle	Description
Resource Owner	L'utilisateur
Client	L'application qui veut accéder aux ressources
Authorization Server	Émet les tokens (Keycloak, Google, Spring Authorization Server)
Resource Server	L'API qui héberge les ressources protégées

Authorization Code Flow + PKCE (le standard actuel)

```
Utilisateur → Client → Authorization Server (page de login)
                        | (l'utilisateur s'authentifie + consent)
                        ← code d'autorisation ┘
Client → échange code (+ code_verifier PKCE) → Authorization Server
                        ← access token (+ id token + refresh) ┘
Client → appelle l'API avec l'access token → Resource Server
```

Flow	Usage	Statut
Authorization Code + PKCE	Web, mobile, SPA	Recommandé
Client Credentials	Machine-à-machine (pas d'utilisateur)	OK pour services
~~Implicit~~	Ancien SPA	Déconseillé (remplacé par Code+PKCE)
~~Password (ROPC)~~	Legacy	Déconseillé

PKCE en deux mots

PKCE (Proof Key for Code Exchange) protège l'échange du code d'autorisation pour les clients publics (mobile/SPA) : le client génère un `code_verifier` secret et envoie son hash (`code_challenge`) au début, puis prouve qu'il le détient à l'échange.

ID token vs Access token

- **ID token** (OIDC) : prouve **qui** est l'utilisateur (claims `sub` , `name` , `email`). Destiné au **client**.
- **Access token** (OAuth2) : autorise l'accès à une **API**. Destiné au **resource server**.

4.2 Spring Security + OAuth2

Client OAuth2 (login « Se connecter avec Google/GitHub/Keycloak »)

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-oauth2-client</artifactId>
</dependency>
```

```
spring:
  security:
    oauth2:
      client:
        registration:
          keycloak:
            client-id: quickbite-app
            client-secret: ${KC_SECRET}
            scope: openid, profile, email
            authorization-grant-type: authorization_code
        provider:
          keycloak:
            issuer-uri: http://localhost:8080/realms/quickbite
```

```
http.oauth2Login(Customizer.withDefaults()); // déclenche le flow OIDC
```

Resource Server (l'API valide les tokens Keycloak)

```
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: http://localhost:8080/realms/quickbite
```

```
http.oauth2ResourceServer(o -> o.jwt(jwt ->
  jwt.jwtAuthenticationConverter(keycloakRolesConverter())));
```

Mapper les rôles Keycloak

Keycloak place les rôles dans `realm_access.roles` (ou `resource_access`). Écrivez un `Converter` qui extrait ces rôles en `GrantedAuthority` (préfixés `ROLE_`) pour que `hasRole()` fonctionne.

4.3 Keycloak

Keycloak est un serveur open source d'**Identity & Access Management** (IAM) : authentification, SSO, fédération, gestion d'utilisateurs et de rôles.

```
# Lancer Keycloak en local (dev)
docker run -p 8080:8080 \
  -e KEYCLOAK_ADMIN=admin -e KEYCLOAK_ADMIN_PASSWORD=admin \
  quay.io/keycloak/keycloak:24.0 start-dev
```

Concept Keycloak	Définition
Realm	Espace isolé regroupant utilisateurs, clients, rôles
Client	Une application enregistrée (front, API...)
Roles	Rôles realm ou rôles client
Claims / Mappers	Données injectées dans les tokens (rôles, attributs)
Policies	Règles d'autorisation fines (authorization services)

Étapes de configuration type : 1. Créer un **realm** `quickbite`. 2. Créer un **client** `quickbite-app` (type `confidential` ou `public` selon le front). 3. Définir les **rôles** (`admin`, `client`) et les assigner aux utilisateurs. 4. Configurer les **mappers** pour exposer les rôles dans le token. 5. Récupérer l'`issuer-uri` et le client-secret pour Spring.

4.4 Atelier — Login OAuth2 + front protégé

Atelier 4 — Durée 3 h

Mission : déléguer l'authentification de QuickBite à Keycloak : l'API devient resource server, un front (Angular/React) se connecte en OAuth2/OIDC et appelle l'API protégée.

Étapes : 1. (30 min) Lancer Keycloak (Docker), créer le realm, le client, les rôles et 2 utilisateurs. 2. (40 min) Configurer l'API en **resource server** ; mapper les rôles Keycloak. 3. (30 min) Tester l'API : obtenir un token via Keycloak (Postman, grant code/password de dev) et appeler `/admin/**`. 4. (50 min) Configurer le **front** (lib OIDC : `angular-auth-oidc-client` ou `oidc-client-ts`) avec Authorization Code + PKCE. 5. (20 min) Le front appelle l'API avec le Bearer ; vérifier que les rôles filtrent l'UI. 6. (10 min) Restitution.

Critères de réussite :

- Le login se fait sur la page Keycloak (SSO).
- L'API valide le token et applique les rôles (200/403).
- Le front affiche/masque la console admin selon le rôle.

Les pièges de l'intégration Keycloak

Surveillez trois points : un `issuer-uri` **incohérent** (localhost vs nom de conteneur), des **horloges désynchronisées** (qui invalident le token), et des **rôles non mappés** (qui donnent un `403` systématique). Réflexe de débogage : collez le token sur jwt.io pour décoder et vérifier ses *claims* et ses rôles.

Quiz — Jour 4

1. OAuth2 vs OIDC ?
2. ID token vs access token ?
3. Quel flow pour une SPA, et pourquoi PKCE ?
4. Qu'est-ce qu'un realm Keycloak ?
5. Où Keycloak place-t-il les rôles dans le token ?

Jour 5 — Sécurité avancée, bonnes pratiques & TP final

🎯 Objectifs du Jour 5

- Configurer correctement CORS et CSRF selon le contexte.
- Mettre en place rate limiting, gestion d'exceptions et audit.
- Sécuriser une architecture microservices (API Gateway, rotation de clés).
- Appliquer une checklist de durcissement OWASP.
- Réaliser un TP final intégrant tout le module.

5.1 Sécurisation réelle d'une application

CORS vs CSRF (à ne pas confondre)

	CORS	CSRF
Quoi	Autoriser un autre domaine à appeler l'API (navigateur)	Empêcher une requête forgée exécutée à l'insu de l'utilisateur
C'est...	Une permission	Une attaque (et sa parade)
Concerne	API consommée par un front d'un autre origin	Sessions/cookies

```
@Bean
CorsConfigurationSource corsSource() {
    CorsConfiguration c = new CorsConfiguration();
    c.setAllowedOrigins(List.of("https://app.quickbite.com"));
    c.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE"));
    c.setAllowedHeaders(List.of("Authorization", "Content-Type"));
    UrlBasedCorsConfigurationSource s = new UrlBasedCorsConfigurationSource();
    s.registerCorsConfiguration("/**", c);
    return s;
}
```

⚠️ Quand désactiver CSRF ?

Pour une API **stateless** consommée par token (pas de cookie de session), CSRF n'est pas pertinent et on le désactive. Pour une app web à **session + cookies**, CSRF doit rester **actif**. Ne désactivez jamais CSRF « par habitude ».

Rate limiting & throttling

Limiter le nombre de requêtes par client/IP pour contrer brute force et abus. En Spring : **Bucket4j**, ou au niveau de l'**API Gateway** / reverse proxy (Nginx, Spring Cloud Gateway).

Gestion d'exceptions (ne pas fuiter d'info)

```
@RestControllerAdvice
public class ApiExceptionHandler {
    @ExceptionHandler(Exception.class)
    public ResponseEntity<ApiError> handle(Exception ex) {
        // Logguer le détail côté serveur, renvoyer un message générique au client
        return ResponseEntity.status(500).body(new ApiError("Erreur interne"));
    }
}
```

Audit & logging (Spring Boot Actuator)

- Tracer les **événements de sécurité** (login réussi/échoué, accès refusé).
- Exposer prudemment les endpoints Actuator (les protéger, ne pas tout ouvrir).
- Ne **jamais** logger de secrets, mots de passe ou tokens.

5.2 Stratégies avancées

Microservices & API Gateway

Une **API Gateway** (Spring Cloud Gateway) centralise : authentification, rate limiting, routage, terminaison TLS. Les services internes font confiance aux tokens validés en amont (ou re-valident en *zero-trust*).

```
Client → API Gateway (AuthN/Z, rate limit, TLS) → Service A
                                           ↳ Service B
                                           ↳ Service C
```

Rotation des clés de signature JWT

Prévoir une **rotation** régulière des clés (JWKS avec plusieurs clés actives + **kid**) pour limiter l'impact d'une compromission, sans invalider brutalement tous les tokens.

Durcissement OWASP — checklist

- [] HTTPS/TLS partout, HSTS, redirection HTTP→HTTPS.
- [] Mots de passe : BCrypt/Argon2 + politique de robustesse.
- [] Tokens : durée courte, rotation des refresh, secret fort.
- [] Validation/sanitization de toutes les entrées (anti-injection).

- [] Headers de sécurité : CSP, X-Content-Type-Options, X-Frame-Options.
- [] CORS restrictif, CSRF cohérent avec le modèle de session.
- [] Rate limiting + logout sur le login.
- [] Moindre privilège (rôles/permissions, comptes de service).
- [] Dépendances à jour (scan SCA), pas de secrets en clair.
- [] Logs/audit sans données sensibles, monitoring des anomalies.

! Référence OWASP Top 10

Cadrez les bonnes pratiques sur l'**OWASP Top 10** (Broken Access Control, Cryptographic Failures, Injection, etc.). « Broken Access Control » est aujourd'hui le risque n°1 — d'où l'importance des Jours 2 et 3.

5.3 Grand TP final

TP final — Durée 4 h — Évalué

Mission : livrer un système complet et sécurisé QuickBite, intégrant tout le module.

Livrables attendus :

-  **API REST sécurisée par JWT** (login/refresh, endpoints protégés).
-  **Console admin / endpoints RBAC** (rôles & permissions fins).
-  **Intégration Keycloak** (authentification + rôles délégués).
-  **Front protégé en OAuth2/OIDC** (démon Angular/React).
-  **Documentation Postman** + note de sécurité (CORS, CSRF, rotation, OWASP).

Déroulé suggéré : 1. (60 min) Finaliser l'authentification (JWT maison **ou** resource server Keycloak — au choix) et le RBAC. 2. (60 min) Durcissement : CORS, gestion d'exceptions, rate limiting sur `/auth/login`, headers de sécurité. 3. (60 min) Front : login OIDC + appel API + affichage conditionnel selon rôle. 4. (40 min) Documentation Postman + checklist OWASP renseignée. 5. (40 min) **Démonstration + Q/R** par équipe.

Grille d'évaluation (20 pts)

Critère	Points
Authentification fonctionnelle (JWT ou Keycloak)	4
Autorisation RBAC correcte (rôles/permissions, 401/403)	4
Intégration OAuth2/OIDC (Keycloak)	4
Durcissement (CORS, exceptions, rate limit, headers)	4
Front protégé + documentation Postman	2
Qualité du code, clarté de la démo	2

Évaluation finale du Module 3

QCM de synthèse

1. AuthN vs AuthZ ; quel code pour chacun (401/403) ?
2. Pourquoi BCrypt/Argon2 et pas SHA-256 seul ?
3. Un JWT est-il confidentiel ? Que garantit la signature ?
4. Access vs refresh token + rotation.
5. OAuth2 vs OIDC ; ID token vs access token.
6. Quel flow pour une SPA et pourquoi PKCE ?
7. Quand désactiver CSRF ? Qu'est-ce que CORS ?
8. Citez 3 mesures de la checklist OWASP de durcissement.

✓ Clôture du cursus complet

Reliez les 3 modules : **Agile** définit *quoi* livrer et *dans quel ordre* ; **DevOps** permet de le livrer *vite et sûrement* ; **Spring Security** garantit que ce qui est livré est *protégé*. Terminez par une rétrospective globale et la remise des grilles d'auto-évaluation des compétences.